

```

# =====
#                               BLOCK PUZZLE GAME
# =====
#
# Computer Graphics Sessional Project
#
# Project Title:
# "Block Puzzle - A 2D Computer Graphics Game Using PyOpenGL"
#
# Programming Language : Python
# Graphics Library      : PyOpenGL
# Window Toolkit        : GLUT
# Platform              : Windows
#
# =====
#                               PROJECT FEATURES
# =====
#
# 1. Start Menu
# 2. 10 x 20 Puzzle Board
# 3. Seven Different Puzzle Blocks
# 4. Falling Blocks
# 5. Left / Right Movement
# 6. Block Rotation
# 7. Soft Drop
# 8. Hard Drop
# 9. Collision Detection
# 10. Boundary Checking
# 11. Complete Line Detection
# 12. Line Clearing
# 13. Score System
# 14. Level System
# 15. Increasing Falling Speed
# 16. Next Block Preview
# 17. Ghost/Landing Position
# 18. Pause / Resume
# 19. Restart
# 20. Game Over
#
# =====
#                               COMPUTER GRAPHICS CONCEPTS
# =====
#
# 1. Line Drawing
# 2. Shape Drawing
# 3. Color Filling
# 4. 2D Translation
# 5. 2D Rotation
# 6. Boundary Clipping / Boundary Checking
# 7. Animation
# 8. Double Buffering
# 9. 2D Orthographic Projection
#
# =====
#                               CONTROLS

```

```

# =====
#
# ENTER      -> Start Game
# LEFT ARROW -> Move Block Left
# RIGHT ARROW -> Move Block Right
# DOWN ARROW -> Soft Drop
# UP ARROW   -> Rotate Block
# SPACE      -> Hard Drop
# P          -> Pause / Resume
# R          -> Restart Game
# ESC        -> Exit
#
# =====

# -----
# IMPORT REQUIRED LIBRARIES
# -----

from OpenGL.GL import *
from OpenGL.GLUT import *
from OpenGL.GLU import *

import random
import sys

# =====
# WINDOW SETTINGS
# =====

# Width of the game window in pixels.
WINDOW_WIDTH = 1000

# Height of the game window in pixels.
WINDOW_HEIGHT = 700

# Title shown on the window.
WINDOW_TITLE = "Block Puzzle - Computer Graphics Project"

# =====
# GAME BOARD SETTINGS
# =====

# Number of columns in the puzzle board.
COLS = 10

# Number of rows in the puzzle board.
ROWS = 20

# Size of each puzzle cell in pixels.
CELL_SIZE = 28

# Starting position of the board.
BOARD_X = 120
BOARD_Y = 70

# Calculate total board width.

```

```

BOARD_WIDTH = COLS * CELL_SIZE

# Calculate total board height.
BOARD_HEIGHT = ROWS * CELL_SIZE

# =====
# GAME STATES
# =====

# Main menu state.
MENU = 0

# Normal gameplay state.
PLAYING = 1

# Pause state.
PAUSED = 2

# Game over state.
GAME_OVER = 3

# Initially the game starts at the menu.
game_state = MENU

# =====
# GAME VARIABLES
# =====

# The board is a 2D list.
#
# None = empty cell
# color = occupied cell
#
# Example:
#
# board[y][x]
#
# This is the main data structure used to store placed blocks.
board = []

# Current falling block.
current_piece = None

# Next block shown in the preview panel.
next_piece = None

# Player score.
score = 0

# Number of completed lines.

```

```

lines_cleared = 0

# Current game level.
level = 1

# Used for automatic falling.
auto_fall_counter = 0


# =====
# COLORS
# =====

# Background color.
BACKGROUND = (0.025, 0.035, 0.065)

# White color.
WHITE = (1.0, 1.0, 1.0)

# Grid color.
GRID_COLOR = (0.10, 0.12, 0.18)

# Board border color.
BORDER_COLOR = (0.30, 0.35, 0.45)

# Side panel background.
PANEL_COLOR = (0.06, 0.08, 0.13)

# Side panel border.
PANEL_BORDER = (0.25, 0.30, 0.40)

# Colors used by puzzle pieces.
CYAN = (0.1, 0.85, 1.0)
YELLOW = (1.0, 0.85, 0.1)
PURPLE = (0.65, 0.25, 1.0)
GREEN = (0.15, 0.9, 0.35)
RED = (1.0, 0.15, 0.20)
BLUE = (0.15, 0.35, 1.0)
ORANGE = (1.0, 0.45, 0.05)


# =====
# TETROMINO DEFINITIONS
# =====
#
# Each puzzle piece consists of four square blocks.
#
# Coordinates are local coordinates.
#
# Example:
#      (-1,0) (0,0) (1,0)

```

```

#
# The piece is later translated to its real position on the board.
#
# =====

SHAPES = [

    # -----
    # I BLOCK
    # -----
    {
        "name": "I",

        "color": CYAN,

        "blocks": [
            (-1, 0),
            (0, 0),
            (1, 0),
            (2, 0)
        ]
    },

    # -----
    # O BLOCK
    # -----
    {
        "name": "O",

        "color": YELLOW,

        "blocks": [
            (0, 0),
            (1, 0),
            (0, 1),
            (1, 1)
        ]
    },

    # -----
    # T BLOCK
    # -----
    {
        "name": "T",

        "color": PURPLE,

        "blocks": [
            (-1, 0),
            (0, 0),
            (1, 0),
            (0, 1)
        ]
    },

    # -----

```

```
# S BLOCK
# -----
{
    "name": "S",

    "color": GREEN,

    "blocks": [
        (0, 0),
        (1, 0),
        (-1, 1),
        (0, 1)
    ]
},

# -----
# Z BLOCK
# -----
{
    "name": "Z",

    "color": RED,

    "blocks": [
        (-1, 0),
        (0, 0),
        (0, 1),
        (1, 1)
    ]
},

# -----
# J BLOCK
# -----
{
    "name": "J",

    "color": BLUE,

    "blocks": [
        (-1, 0),
        (-1, 1),
        (0, 1),
        (1, 1)
    ]
},

# -----
# L BLOCK
# -----
{
    "name": "L",

    "color": ORANGE,

    "blocks": [
```

```

        (1, 0),
        (-1, 1),
        (0, 1),
        (1, 1)
    ]
}
]

```

```

# =====
# CREATE EMPTY BOARD
# =====

```

```

def create_board():

    global board

    # Create an empty list.
    board = []

    # Create 20 rows.
    for y in range(ROWS):

        row = []

        # Create 10 columns in each row.
        for x in range(COLS):

            # None means the cell is empty.
            row.append(None)

        # Add the row to the board.
        board.append(row)

```

```

# =====
# CREATE A NEW PUZZLE PIECE
# =====

```

```

def create_piece(shape=None):

    # If no shape was provided,
    # randomly select one of the seven pieces.
    if shape is None:

        shape = random.choice(SHAPES)

    # Create the piece object.
    piece = {

        # Name of the piece.
        "name": shape["name"],

```

```

    # Color of the piece.
    "color": shape["color"],

    # Local coordinates of its four blocks.
    "blocks": list(shape["blocks"]),

    # Start approximately at the middle of the board.
    "x": COLS // 2,

    # Start near the top of the board.
    "y": ROWS - 2,

    # Initial rotation.
    "rotation": 0
}

return piece

```

```

# =====
# 2D ROTATION
# =====

```

```
def rotate_point(x, y):
```

```

    # -----
    # COMPUTER GRAPHICS: 2D ROTATION
    # -----
    #
    # General rotation equations:
    #
    #  $x' = x \cos(\theta) - y \sin(\theta)$ 
    #  $y' = x \sin(\theta) + y \cos(\theta)$ 
    #
    # For 90-degree rotation:
    #
    #  $\cos(90) = 0$ 
    #  $\sin(90) = 1$ 
    #
    # Therefore:
    #
    #  $x' = -y$ 
    #  $y' = x$ 
    #
    # This function performs one 90-degree rotation.
    # -----

    new_x = -y
    new_y = x

    return new_x, new_y

```

```
# =====
# GET ROTATED BLOCK COORDINATES
# =====
```

```
def get_rotated_blocks(piece):
```

```
    # List to store rotated coordinates.
    blocks = []
```

```
    # Get original local coordinates.
    for x, y in piece["blocks"]:
```

```
        # Start with the original point.
        rx = x
        ry = y
```

```
        # Apply 90-degree rotation as many times as necessary.
        for i in range(piece["rotation"]):
```

```
            rx, ry = rotate_point(rx, ry)
```

```
        # Store the rotated coordinate.
        blocks.append((rx, ry))
```

```
    return blocks
```

```
# =====
# GET ACTUAL BOARD CELLS OF A PIECE
# =====
```

```
def get_piece_cells(piece):
```

```
    cells = []
```

```
    # Get coordinates after rotation.
    blocks = get_rotated_blocks(piece)
```

```
    for bx, by in blocks:
```

```
        # -----
        # COMPUTER GRAPHICS: 2D TRANSLATION
        # -----
        #
        # Local coordinates are moved to the actual board position.
        #
        #  $x' = x + T_x$ 
        #  $y' = y + T_y$ 
        #
        # Here:
        #
        #  $T_x = \text{piece}["x"]$ 
```

```

    # Ty = piece["y"]
    #
    # -----

    x = piece["x"] + bx
    y = piece["y"] + by

    cells.append((x, y))

return cells

```

```

# =====
# COLLISION DETECTION
# =====

```

```
def collision(piece, dx=0, dy=0, rotation=None):
```

```
    # Create a temporary test piece.
```

```
    test_piece = {
```

```
        "name": piece["name"],
```

```
        "color": piece["color"],
```

```
        "blocks": piece["blocks"],
```

```
        # Test position.
```

```
        "x": piece["x"] + dx,
```

```
        "y": piece["y"] + dy,
```

```
        # Keep current rotation.
```

```
        "rotation": piece["rotation"]
```

```
    }
```

```
    # If a different rotation was supplied,
```

```
    # use that rotation for the test.
```

```
    if rotation is not None:
```

```
        test_piece["rotation"] = rotation
```

```
    # Get all cells occupied by the test piece.
```

```
    cells = get_piece_cells(test_piece)
```

```
    for x, y in cells:
```

```
        # -----
```

```
        # BOUNDARY CHECKING / CLIPPING
```

```
        # -----
```

```
        #
```

```
        # A block cannot move beyond the left side.
```

```
        # -----
```

```

    if x < 0:

        return True

    # Cannot move beyond the right side.
    if x >= COLS:

        return True

    # Cannot go below the bottom.
    if y < 0:

        return True

    # If the block is above the board,
    # it is temporarily allowed.
    if y >= ROWS:

        continue

    # -----
    # COLLISION DETECTION
    # -----
    #
    # If this cell is already occupied,
    # collision occurs.
    # -----

    if board[y][x] is not None:

        return True

    # No collision found.
    return False


# =====
# MOVE PIECE
# =====

def move_piece(dx, dy):

    global current_piece

    # If there is no current piece,
    # there is nothing to move.
    if current_piece is None:

        return False

    # Check whether movement is safe.
    if not collision(current_piece, dx, dy):

```

```

# -----
# 2D TRANSLATION
# -----
#
# Change the position of the piece.
# -----

current_piece["x"] += dx
current_piece["y"] += dy

return True

return False

# =====
# ROTATE PIECE
# =====

def rotate_piece():

    global current_piece

    if current_piece is None:

        return

    # Calculate next rotation.
    new_rotation = (
        current_piece["rotation"] + 1
    ) % 4

    # -----
    # Test normal rotation first.
    # -----

    if not collision(
        current_piece,
        0,
        0,
        new_rotation
    ):

        # Apply rotation.
        current_piece["rotation"] = new_rotation

    else:

        # -----
        # SIMPLE WALL KICK
        # -----
        #
        # If rotation hits a wall,
        # try moving one cell left.

```

```

# -----

if not collision(
    current_piece,
    -1,
    0,
    new_rotation
):

    current_piece["x"] -= 1

    current_piece["rotation"] = new_rotation

# If left does not work,
# try moving one cell right.
elif not collision(
    current_piece,
    1,
    0,
    new_rotation
):

    current_piece["x"] += 1

    current_piece["rotation"] = new_rotation


# =====
# LOCK PIECE INTO BOARD
# =====

def lock_piece():

    global current_piece

    # Get final board positions.
    cells = get_piece_cells(current_piece)

    for x, y in cells:

        # Make sure the coordinate is inside the board.
        if (
            0 <= x < COLS
            and
            0 <= y < ROWS
        ):

            # Store the block color in the board.
            board[y][x] = current_piece["color"]

```

```

# =====
# FIND FULL ROWS
# =====

def find_full_rows():

    full_rows = []

    # Check every row.
    for y in range(ROWS):

        full = True

        # Check every cell.
        for x in range(COLS):

            if board[y][x] is None:

                full = False

                break

        # If every cell was occupied,
        # this is a complete row.
        if full:

            full_rows.append(y)

    return full_rows


# =====
# CLEAR COMPLETE LINES
# =====

def clear_lines():

    global score
    global lines_cleared
    global level

    # Find complete rows.
    rows = find_full_rows()

    # Nothing to clear.
    if not rows:

        return

    # Remove complete rows.
    for y in sorted(rows, reverse=True):

```

```

        del board[y]

        # Add an empty row at the top.
        board.append(
            [None for _ in range(COLS)]
        )

    # Number of lines removed.
    number = len(rows)

    # -----
    # SCORE SYSTEM
    # -----

    if number == 1:

        score += 100 * level

    elif number == 2:

        score += 300 * level

    elif number == 3:

        score += 500 * level

    elif number == 4:

        score += 800 * level

    # Update line count.
    lines_cleared += number

    # Every 10 lines increases the level.
    level = (
        lines_cleared // 10
    ) + 1

# =====
# SPAWN NEXT PIECE
# =====

def spawn_piece():

    global current_piece
    global next_piece
    global game_state

    # If there is no next piece,
    # create both current and next pieces.
    if next_piece is None:

        current_piece = create_piece()

```

```

        next_piece = create_piece()
    else:

        # Move preview piece to current position.
        current_piece = next_piece

        # Generate a new preview piece.
        next_piece = create_piece()

# -----
# GAME OVER CHECK
# -----
#
# If the new piece cannot be placed,
# the board is full and the game ends.
# -----

    if collision(current_piece):

        game_state = GAME_OVER

# =====
# HARD DROP
# =====

def hard_drop():

    global score

    # Distance moved downward.
    distance = 0

    # Keep moving downward until collision.
    while move_piece(0, -1):

        distance += 1

    # Give extra score for hard drop.
    score += distance * 2

    # Place the block permanently.
    lock_piece()

    # Remove completed rows.
    clear_lines()

    # Generate next block.
    spawn_piece()

# =====

```

```

# SOFT DROP
# =====

def soft_drop():

    global score

    # Try to move down one cell.
    if move_piece(0, -1):

        # Small score for manually dropping.
        score += 1

    else:

        # If movement is impossible,
        # lock the block.
        lock_piece()

        # Check for complete rows.
        clear_lines()

        # Spawn the next piece.
        spawn_piece()

# =====
# RESTART GAME
# =====

def restart_game():

    global score
    global lines_cleared
    global level
    global game_state
    global current_piece
    global next_piece
    global auto_fall_counter

    # Create empty board.
    create_board()

    # Reset score.
    score = 0

    # Reset lines.
    lines_cleared = 0

    # Reset level.
    level = 1

    # Remove current piece.
    current_piece = None

```

```

# Remove next piece.
next_piece = None

# Reset automatic falling timer.
auto_fall_counter = 0

# Change game state.
game_state = PLAYING

# Create first piece.
spawn_piece()

# =====
# DRAW FILLED RECTANGLE
# =====
def draw_rectangle(
    x,
    y,
    width,
    height,
    color
):
    # Set rectangle color.
    glColor3f(
        color[0],
        color[1],
        color[2]
    )
    # -----
    # COMPUTER GRAPHICS: SHAPE DRAWING
    # -----
    #
    # GL_QUADS draws a filled four-sided polygon.
    #
    # It is used for:
    # - blocks
    # - panels
    # - backgrounds
    # - buttons
    # -----

    glBegin(GL_QUADS)

    glVertex2f(
        x,
        y
    )
    glVertex2f(
        x + width,
        y
    )
    glVertex2f(
        x + width,

```

```

        y + height
    )
    glVertex2f(
        x,
        y + height
    )
    glEnd()

# =====
# DRAW RECTANGLE BORDER
# =====

def draw_rectangle_outline(
    x,
    y,
    width,
    height,
    color,
    line_width=2
):

    # Set border color.
    glColor3f(
        color[0],
        color[1],
        color[2]
    )

    # Set line thickness.
    glLineWidth(line_width)

    # -----
    # COMPUTER GRAPHICS: LINE DRAWING
    # -----

    glBegin(GL_LINE_LOOP)

    glVertex2f(
        x,
        y
    )

    glVertex2f(
        x + width,
        y
    )
    glVertex2f(
        x + width,
        y + height
    )
    glVertex2f(
        x,
        y + height
    )

```

```

        glEnd()

# =====
# DRAW TEXT
# =====

def draw_text(
    x,
    y,
    text,
    font=GLUT_BITMAP_HELVETICA_18
):
    # Text color.
    glColor3f(
        WHITE[0],
        WHITE[1],
        WHITE[2]
    )

    # Set text starting position.
    glRasterPos2f(
        x,
        y
    )

    # Draw every character.
    for character in text:
        glutBitmapCharacter(
            font,
            ord(character)
        )

# =====
# DRAW CENTERED TEXT
# =====

def draw_center_text(
    y,
    text,
    font=GLUT_BITMAP_HELVETICA_18
):
    # Calculate total text width.
    width = 0

    for character in text:
        width += glutBitmapWidth(
            font,
            ord(character)
        )

    # Calculate centered X position.

```

```

    x = (
        WINDOW_WIDTH - width
    ) / 2

    # Draw the text.
    draw_text(
        x,
        y,
        text,
        font
    )

# =====
# DRAW GAME BOARD
# =====

def draw_board():

    # -----
    # Draw dark board background.
    # -----

    draw_rectangle(
        BOARD_X,
        BOARD_Y,
        BOARD_WIDTH,
        BOARD_HEIGHT,
        (0.04, 0.05, 0.09)
    )

    # -----
    # COMPUTER GRAPHICS: GRID LINE DRAWING
    # -----

    glColor3f(
        GRID_COLOR[0],
        GRID_COLOR[1],
        GRID_COLOR[2]
    )

    glLineWidth(1)

    glBegin(GL_LINES)

    # Vertical grid lines.
    for x in range(COLS + 1):

        px = (
            BOARD_X
            +
            x * CELL_SIZE
        )

        glVertex2f(
            px,

```

```

        BOARD_Y
    )

    glVertex2f(
        px,
        BOARD_Y + BOARD_HEIGHT
    )

# Horizontal grid lines.
for y in range(ROWS + 1):

    py = (
        BOARD_Y
        +
        y * CELL_SIZE
    )

    glVertex2f(
        BOARD_X,
        py
    )

    glVertex2f(
        BOARD_X + BOARD_WIDTH,
        py
    )

glEnd()

# Draw outer border.
draw_rectangle_outline(
    BOARD_X,
    BOARD_Y,
    BOARD_WIDTH,
    BOARD_HEIGHT,
    BORDER_COLOR,
    3
)

# =====
# DRAW ONE PUZZLE CELL
# =====

def draw_cell(
    x,
    y,
    color,
    alpha=1.0
):

    # Convert board coordinate into screen coordinate.
    px = (
        BOARD_X
        +

```

```

        x * CELL_SIZE
    )

    py = (
        BOARD_Y
        +
        y * CELL_SIZE
    )

    # -----
    # MAIN COLOR
    # -----

    glColor4f(
        color[0],
        color[1],
        color[2],
        alpha
    )

    # Draw filled cell.
    glBegin(GL_QUADS)

    glVertex2f(
        px + 2,
        py + 2
    )

    glVertex2f(
        px + CELL_SIZE - 2,
        py + 2
    )

    glVertex2f(
        px + CELL_SIZE - 2,
        py + CELL_SIZE - 2
    )

    glVertex2f(
        px + 2,
        py + CELL_SIZE - 2
    )

    glEnd()

    # -----
    # BLOCK HIGHLIGHT
    #
    # A small bright area gives the block a polished appearance.
    # -----

    glColor4f(
        1.0,
        1.0,
        1.0,
        alpha * 0.25
    )

```

```

    )

    glBegin(GL_QUADS)

    glVertex2f(
        px + 3,
        py + CELL_SIZE - 7
    )

    glVertex2f(
        px + CELL_SIZE - 3,
        py + CELL_SIZE - 7
    )

    glVertex2f(
        px + CELL_SIZE - 3,
        py + CELL_SIZE - 3
    )

    glVertex2f(
        px + 3,
        py + CELL_SIZE - 3
    )

    glEnd()

    # -----
    # BLOCK BORDER
    # -----

    glColor4f(
        0.0,
        0.0,
        0.0,
        alpha * 0.45
    )

    glLineWidth(1)

    glBegin(GL_LINE_LOOP)

    glVertex2f(
        px + 2,
        py + 2
    )

    glVertex2f(
        px + CELL_SIZE - 2,
        py + 2
    )

    glVertex2f(
        px + CELL_SIZE - 2,
        py + CELL_SIZE - 2
    )

```

```

    glVertex2f(
        px + 2,
        py + CELL_SIZE - 2
    )

    glEnd()

# =====
# DRAW ALL LOCKED BLOCKS
# =====

def draw_locked_blocks():

    # Visit every row.
    for y in range(ROWS):

        # Visit every column.
        for x in range(COLS):

            # If the cell contains a block,
            # draw it.
            if board[y][x] is not None:

                draw_cell(
                    x,
                    y,
                    board[y][x]
                )

# =====
# DRAW CURRENT FALLING PIECE
# =====

def draw_current_piece():

    # Nothing to draw if no piece exists.
    if current_piece is None:

        return

    # Get actual board coordinates.
    cells = get_piece_cells(
        current_piece
    )

    # Draw all four blocks.
    for x, y in cells:

        if (
            0 <= x < COLS
            and

```

```

        0 <= y < ROWS
    ):

        draw_cell(
            x,
            y,
            current_piece["color"]
        )

# =====
# FIND GHOST PIECE POSITION
# =====

def get_ghost_y():
    if current_piece is None:
        return 0

    # Start from current Y.
    test_y = current_piece["y"]

    # Continue moving downward.
    while True:
        test_piece = {
            "name": current_piece["name"],
            "color": current_piece["color"],
            "blocks": current_piece["blocks"],
            "x": current_piece["x"],
            "y": test_y,
            "rotation": current_piece["rotation"]
        }

        # Check whether it can move down.
        if collision(
            test_piece,
            0,
            -1
        ):
            break

        # Move test position down.
        test_y -= 1

    return test_y

```

```

# =====
# DRAW GHOST PIECE
# =====

def draw_ghost_piece():

    if current_piece is None:

        return

    # Find landing Y coordinate.
    ghost_y = get_ghost_y()

    # Get rotated block coordinates.
    blocks = get_rotated_blocks(
        current_piece
    )

    # Draw each ghost cell.
    for bx, by in blocks:

        x = (
            current_piece["x"]
            +
            bx
        )

        y = (
            ghost_y
            +
            by
        )

        if (
            0 <= x < COLS
            and
            0 <= y < ROWS
        ):

            # Convert to screen coordinates.
            px = (
                BOARD_X
                +
                x * CELL_SIZE
            )

            py = (
                BOARD_Y
                +
                y * CELL_SIZE
            )

            # Transparent color.

```

```

        glColor4f(
            current_piece["color"][0],
            current_piece["color"][1],
            current_piece["color"][2],
            0.20
        )

        # Draw transparent cell.
        glBegin(GL_QUADS)

        glVertex2f(
            px + 4,
            py + 4
        )

        glVertex2f(
            px + CELL_SIZE - 4,
            py + 4
        )

        glVertex2f(
            px + CELL_SIZE - 4,
            py + CELL_SIZE - 4
        )

        glVertex2f(
            px + 4,
            py + CELL_SIZE - 4
        )

        glEnd()

# =====
# DRAW NEXT PIECE PREVIEW
# =====

def draw_next_piece():

    if next_piece is None:

        return

    # Preview panel position.
    panel_x = 570
    panel_y = 350

    # Draw panel.
    draw_rectangle(
        panel_x,
        panel_y,
        260,
        140,
        PANEL_COLOR
    )

```

```

# Panel border.
draw_rectangle_outline(
    panel_x,
    panel_y,
    260,
    140,
    PANEL_BORDER,
    2
)

# Panel title.
draw_text(
    panel_x + 20,
    panel_y + 105,
    "NEXT BLOCK"
)

# Get block coordinates.
blocks = get_rotated_blocks(
    next_piece
)

# Preview cell size.
preview_cell = 25

# Preview center.
center_x = panel_x + 130
center_y = panel_y + 50

# Draw each preview block.
for bx, by in blocks:

    px = (
        center_x
        +
        bx * preview_cell
    )

    py = (
        center_y
        +
        by * preview_cell
    )

    draw_rectangle(
        px,
        py,
        preview_cell - 3,
        preview_cell - 3,
        next_piece["color"]
    )

# =====

```

```

# DRAW GAME INFORMATION PANEL
# =====

def draw_panel():

    # Panel position.
    panel_x = 570
    panel_y = 70

    panel_width = 260
    panel_height = 250

    # Draw panel background.
    draw_rectangle(
        panel_x,
        panel_y,
        panel_width,
        panel_height,
        PANEL_COLOR
    )

    # Draw panel border.
    draw_rectangle_outline(
        panel_x,
        panel_y,
        panel_width,
        panel_height,
        PANEL_BORDER,
        2
    )

    # Panel heading.
    draw_text(
        panel_x + 20,
        panel_y + 210,
        "GAME INFORMATION"
    )

    # Score label.
    draw_text(
        panel_x + 20,
        panel_y + 170,
        "SCORE"
    )

    # Score value.
    draw_text(
        panel_x + 150,
        panel_y + 170,
        str(score)
    )

    # Lines label.
    draw_text(
        panel_x + 20,
        panel_y + 135,

```

```

        "LINES"
    )

    # Lines value.
    draw_text(
        panel_x + 150,
        panel_y + 135,
        str(lines_cleared)
    )

    # Level label.
    draw_text(
        panel_x + 20,
        panel_y + 100,
        "LEVEL"
    )

    # Level value.
    draw_text(
        panel_x + 150,
        panel_y + 100,
        str(level)
    )

    # Controls.
    draw_text(
        panel_x + 20,
        panel_y + 55,
        "CONTROLS"
    )

    draw_text(
        panel_x + 20,
        panel_y + 30,
        "Arrow Keys / SPACE / P"
    )

# =====
# DRAW TITLE
# =====

def draw_title():

    draw_center_text(
        660,
        "BLOCK PUZZLE",
        GLUT_BITMAP_TIMES_ROMAN_24
    )

    draw_center_text(
        630,
        "COMPUTER GRAPHICS PROJECT"
    )

```

```

# =====
# DRAW MAIN MENU
# =====

def draw_menu():

    # Draw background.
    draw_rectangle(
        0,
        0,
        WINDOW_WIDTH,
        WINDOW_HEIGHT,
        BACKGROUND
    )

    # Main title.
    draw_center_text(
        500,
        "BLOCK PUZZLE",
        GLUT_BITMAP_TIMES_ROMAN_24
    )

    # Subtitle.
    draw_center_text(
        450,
        "A PYOPENGL COMPUTER GRAPHICS GAME"
    )

    # Start instruction.
    draw_center_text(
        350,
        "PRESS ENTER TO START",
        GLUT_BITMAP_HELVETICA_18
    )

    # Controls.
    draw_center_text(
        300,
        "ARROW KEYS = MOVE / ROTATE"
    )

    draw_center_text(
        265,
        "SPACE = HARD DROP"
    )

    draw_center_text(
        230,
        "P = PAUSE"
    )

    draw_center_text(

```

```

        195,
        "R = RESTART"
    )

    draw_center_text(
        160,
        "ESC = EXIT"
    )

    # Footer.
    draw_center_text(
        80,
        "Computer Graphics Sessional"
    )

# =====
# DRAW PAUSE SCREEN
# =====

def draw_pause_screen():

    # Semi-transparent black overlay.
    glColor4f(
        0.0,
        0.0,
        0.0,
        0.65
    )

    glBegin(GL_QUADS)

    glVertex2f(
        0,
        0
    )

    glVertex2f(
        WINDOW_WIDTH,
        0
    )

    glVertex2f(
        WINDOW_WIDTH,
        WINDOW_HEIGHT
    )

    glVertex2f(
        0,
        WINDOW_HEIGHT
    )

    glEnd()

```

```

# Pause text.
draw_center_text(
    380,
    "GAME PAUSED",
    GLUT_BITMAP_TIMES_ROMAN_24
)

draw_center_text(
    330,
    "PRESS P TO RESUME"
)


# =====
# DRAW GAME OVER SCREEN
# =====

def draw_game_over():

    # Semi-transparent black overlay.
    glColor4f(
        0.0,
        0.0,
        0.0,
        0.70
    )

    glBegin(GL_QUADS)

    glVertex2f(
        0,
        0
    )

    glVertex2f(
        WINDOW_WIDTH,
        0
    )

    glVertex2f(
        WINDOW_WIDTH,
        WINDOW_HEIGHT
    )

    glVertex2f(
        0,
        WINDOW_HEIGHT
    )

    glEnd()

    # Game-over message.
    draw_center_text(
        430,

```

```

        "GAME OVER",
        GLUT_BITMAP_TIMES_ROMAN_24
    )

    # Final score.
    draw_center_text(
        380,
        "FINAL SCORE: " + str(score)
    )

    # Final lines.
    draw_center_text(
        345,
        "LINES: " + str(lines_cleared)
    )

    # Restart instruction.
    draw_center_text(
        280,
        "PRESS R TO PLAY AGAIN"
    )

    # Exit instruction.
    draw_center_text(
        245,
        "PRESS ESC TO EXIT"
    )

# =====
# DISPLAY FUNCTION
# =====

def display():

    # Clear previous frame.
    glClear(
        GL_COLOR_BUFFER_BIT
    )

    # -----
    # MENU
    # -----

    if game_state == MENU:

        draw_menu()

    else:

        # -----
        # GAME BACKGROUND
        # -----

        draw_rectangle(

```

```

        0,
        0,
        WINDOW_WIDTH,
        WINDOW_HEIGHT,
        BACKGROUND
    )

    # Game title.
    draw_title()

    # Puzzle board.
    draw_board()

    # Already placed blocks.
    draw_locked_blocks()

    # Ghost/landing position.
    draw_ghost_piece()

    # Current falling block.
    draw_current_piece()

    # Information panel.
    draw_panel()

    # Next block preview.
    draw_next_piece()

    # Pause overlay.
    if game_state == PAUSED:

        draw_pause_screen()

    # Game-over overlay.
    elif game_state == GAME_OVER:

        draw_game_over()

# -----
# DOUBLE BUFFERING
# -----
#
# Instead of displaying directly to the screen,
# GLUT draws the frame in a back buffer and then swaps it.
#
# This reduces flickering during animation.
# -----

glutSwapBuffers()

# =====
# AUTOMATIC GAME TIMER
# =====

```

```

def timer(value):

    # If the game is running,
    # update automatic falling.
    if game_state == PLAYING:

        # -----
        # GAME SPEED
        # -----
        #
        # As level increases,
        # falling interval becomes smaller.
        #
        # This makes the game progressively harder.
        # -----

        interval = max(
            80,
            650 - (level - 1) * 50
        )

        soft_drop_timer(
            interval
        )

    # Ask GLUT to redraw the window.
    glutPostRedisplay()

    # Call this timer again after 16 milliseconds.
    glutTimerFunc(
        16,
        timer,
        0
    )

# =====
# AUTOMATIC FALLING
# =====

def soft_drop_timer(interval):

    global auto_fall_counter

    # Increase timer by 16 milliseconds.
    auto_fall_counter += 16

    # If enough time has passed,
    # move the block downward.
    if auto_fall_counter >= interval:

        # Reset timer.

```

```

    auto_fall_counter = 0

    if current_piece is not None:

        # Try to move one cell downward.
        if not move_piece(0, -1):

            # Cannot move further.
            # Lock the piece.
            lock_piece()

            # Remove complete lines.
            clear_lines()

            # Create next piece.
            spawn_piece()

# =====
# NORMAL KEYBOARD INPUT
# =====

def keyboard(key, x, y):

    global game_state

    # Convert byte to character.
    key = key.decode(
        "utf-8"
    ).lower()

    # -----
    # ESCAPE
    # -----

    if key == "\x1b":

        sys.exit()

    # -----
    # ENTER
    # -----

    if key == "\r":

        if game_state == MENU:

            restart_game()

        return

    # -----
    # RESTART
    # -----

```

```

    if key == "r":
        restart_game()

        return

# -----
# PAUSE
# -----

    if key == "p":
        if game_state == PLAYING:
            game_state = PAUSED

        elif game_state == PAUSED:
            game_state = PLAYING

        return

# -----
# HARD DROP
# -----

    if key == " ":
        if game_state == PLAYING:
            hard_drop()

        return

# =====
# SPECIAL KEYBOARD INPUT
# =====

def special_keys(key, x, y):
    # Ignore movement when not playing.
    if game_state != PLAYING:
        return

# -----
# LEFT ARROW
# -----

    if key == GLUT_KEY_LEFT:
        move_piece(
            -1,

```

```

        0
    )

# -----
# RIGHT ARROW
# -----

elif key == GLUT_KEY_RIGHT:

    move_piece(
        1,
        0
    )

# -----
# DOWN ARROW
# -----

elif key == GLUT_KEY_DOWN:

    soft_drop()

# -----
# UP ARROW
# -----

elif key == GLUT_KEY_UP:

    rotate_piece()

# Request screen redraw.
glutPostRedisplay()

# =====
# MOUSE INPUT
# =====

def mouse(button, state, x, y):

    # This project is primarily keyboard controlled.
    #
    # GLUT still provides mouse event handling.
    # It can be expanded later with graphical buttons.

    if button == GLUT_LEFT_BUTTON:

        if state == GLUT_DOWN:

            # Currently no mouse action.
            pass

# =====
# WINDOW RESIZE FUNCTION
# =====

```

```

def reshape(width, height):

    # Define viewport.
    glViewport(
        0,
        0,
        width,
        height
    )

    # Select projection matrix.
    glMatrixMode(
        GL_PROJECTION
    )

    # Reset matrix.
    glLoadIdentity()

    # -----
    # COMPUTER GRAPHICS:
    # 2D ORTHOGRAPHIC PROJECTION
    # -----
    #
    # gluOrtho2D creates a 2D coordinate system.
    #
    # X: 0 -> 1000
    # Y: 0 -> 700
    #
    # No perspective is required because this is a 2D game.
    # -----

    gluOrtho2D(
        0,
        WINDOW_WIDTH,
        0,
        WINDOW_HEIGHT
    )

    # Select model-view matrix.
    glMatrixMode(
        GL_MODELVIEW
    )

    # Reset it.
    glLoadIdentity()

    # =====
    # OPENGGL INITIALIZATION
    # =====

def init_opengl():

    # Set background color.
    glClearColor(
        BACKGROUND[0],

```

```

        BACKGROUND[1],
        BACKGROUND[2],
        1.0
    )

# -----
# ALPHA BLENDING
# -----
#
# Required for:
# - ghost piece
# - pause overlay
# - game-over overlay
#
# -----

glEnable(
    GL_BLEND
)

glBlendFunc(
    GL_SRC_ALPHA,
    GL_ONE_MINUS_SRC_ALPHA
)

# This is a 2D game,
# so depth testing is unnecessary.
glDisable(
    GL_DEPTH_TEST
)

# Select projection matrix.
glMatrixMode(
    GL_PROJECTION
)

# Reset projection matrix.
glLoadIdentity()

# Set 2D coordinate system.
gluOrtho2D(
    0,
    WINDOW_WIDTH,
    0,
    WINDOW_HEIGHT
)

# Select model-view matrix.
glMatrixMode(
    GL_MODELVIEW
)

# Reset model-view matrix.
glLoadIdentity()

# =====

```

```

# MAIN FUNCTION
# =====

def main():

    # Randomize puzzle-piece selection.
    random.seed()

    # Create empty game board.
    create_board()

    # Initialize GLUT.
    glutInit(
        sys.argv
    )

    # -----
    # DOUBLE BUFFER + RGBA
    # -----

    glutInitDisplayMode(
        GLUT_DOUBLE |
        GLUT_RGBA
    )

    # Set window size.
    glutInitWindowSize(
        WINDOW_WIDTH,
        WINDOW_HEIGHT
    )

    # Set initial window position.
    glutInitWindowPosition(
        100,
        50
    )

    # Create the OpenGL window.
    glutCreateWindow(
        WINDOW_TITLE.encode()
    )

    # Initialize OpenGL.
    init_opengl()

    # Register display callback.
    glutDisplayFunc(
        display
    )

    # Register resize callback.
    glutReshapeFunc(
        reshape
    )

    # Register normal keyboard callback.

```

```

glutKeyboardFunc(
    keyboard
)

# Register special-key callback.
glutSpecialFunc(
    special_keys
)

# Register mouse callback.
glutMouseFunc(
    mouse
)

# Start game timer.
glutTimerFunc(
    16,
    timer,
    0
)

# Start GLUT event loop.
glutMainLoop()

# =====
# PROGRAM ENTRY POINT
# =====

if __name__ == "__main__":

    main()

# =====
#                               END OF PROJECT
# =====

```